

Prints the
second half

```

fon (int i = n; i >= 1; i--) {
    fon (int j = 1; j <= (n-i); j++) { // Space
        cout << " "; // Single Space
    }
    fon (int k = 1; k <= i; k++) { // Stars
        cout << "* ";
    }
    cout << endl;
}
return 0;
}

```

• OPERATORS :-

A Symbol that tells the compiler to perform a specific operation on one or more operands.

(A) Binary Operators → works on two operands

Ex → $a+b$, $a-b$, $a*b$, a/b , $a \% b$ (Modulo)

(B) Unary Operator → works on one operand

EX:- Post Increment ⇒ $a++$

Post Decrement ⇒ $a--$

Pre Increment ⇒ $++a$

Pre decrement ⇒ $--a$

(C) Comparison Operator → Compares two operands, they either return true(1) or false(0)

Ex: → $==$, $!=$, $<$, $>$, $<=$, $>=$

(D) Logical Operators → These operators are used to combine or manipulate two or more conditions. They return true(1)

on false (0).

Ex:- $\&\&$ (AND), $\parallel$ (OR), $!$ (NOT)

AND		
A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR		
A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

NOT	
A	B
0	1
1	0

Ex:- $2 \&\& 3 \rightarrow \text{Output} = 1$
 $0 \&\& 3 \rightarrow \text{Output} = 0$
 $0 \&\& 0 \rightarrow \text{Output} = 0$

} Treats any non-zero number as 1

$2 \parallel 5 \rightarrow \text{Output} = 1$

$0 \parallel 5 \rightarrow \text{Output} = 0$

$0 \parallel 0 \rightarrow \text{Output} = 0$

$! 10 = 0$
 $! 5 = 0$
 $! 0 = 1$

} Treats non-zero numbers as 1

(E) Bitwise Operators $\rightarrow$ Bitwise Operators are used to perform operations on the binary (bit-level) representation of integer values.

(i) Bitwise AND $\Rightarrow \&$

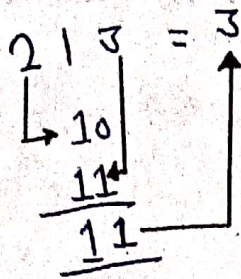
$2 \& 3 = 2$

(Binary)

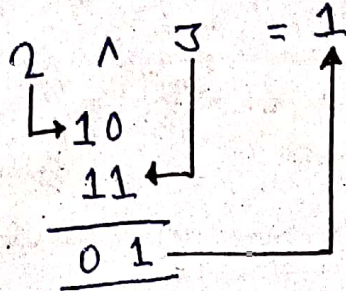
$\begin{array}{r} 10 \\ 11 \\ \hline 10 \end{array}$

And operation takes place

(ii) Bitwise OR $\Rightarrow$



(iii) Bitwise XOR $\Rightarrow$



(XOR)

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

(iv) Bitwise NOT $\Rightarrow$

~~num = ~num~~

$\sim \text{num} = -\text{num} - 1$

$\sim 10 = -11$

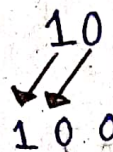
$\sim -10 = 9$

(v) Left Shift $\Rightarrow$

$\text{num} \ll x = \text{num} \times 2^x$

Ex $\rightarrow 2 \ll 1$

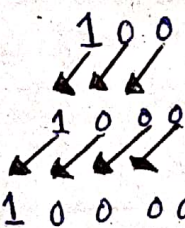
Binary form $\Rightarrow$



$\Rightarrow$ It is equivalent to 4 in decimal.

So, $2 \ll 1 = 4$

Ex $\rightarrow 4 \ll 2$



$\Rightarrow$ It is equivalent to 16

So,

$4 \ll 2 = 16$

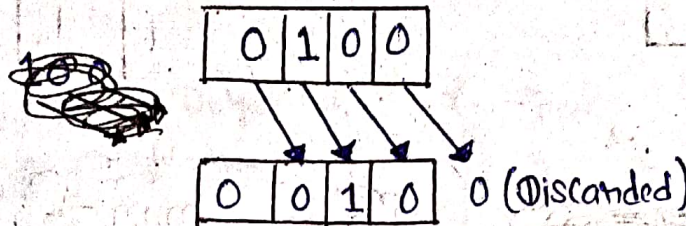
NOTE → left Shift of a negative number is not defined,
 $\text{num} \neq -ve$
 ex → $-6 \ll 2$ Invalid

(vi) Right Shift $\Rightarrow \gg$

$$\text{num} \gg x = \frac{\text{num}}{2^x}$$

Ex:-

4 $\gg$ 1



0010 → It is equivalent to 2

So,

$$4 \gg 1 = 2$$

(F) Assignment Operator

$a = 3 \Rightarrow$ Assigns 3 to variable a

$a += 3 \Rightarrow a = a + 3$

$a -= 3 \Rightarrow a = a - 3$

$a *= 3 \Rightarrow a = a * 3$

$a /= 3 \Rightarrow a = a / 3$

$a \% = 3 \Rightarrow a = a \% 3$

Ex:-

3 > 2 > 1

True (1)

1 > 1 → False (0)

So, it returns False (0)

Operator Precedence: -

Operator		Precedence
() , [], . , -> ++ , --		Left to Right
++a , --a , + , - , ! , ~ , (type) , * & , sizeof	Unary	Right to left
* , / , %	Arithmetic	Left to Right
+ , -	Arithmetic	Left to Right
<< , >>	Shift	Left to Right
< , <= , > , >=	Comparison	Left to Right
== , !=	Equal to , not equal to	Left to Right
&	Bitwise AND	Left to Right
^	Bitwise XOR	Left to Right
	Bitwise OR	left to Right
&&	Logical AND	Left to Right
	Logical OR	Left to Right
?:	Ternary	Right to left
= , += , -= , *= , /= , % = , &= , = , = , << = , >> =	Assignment	Right to left